Mechanistic Signal Compression: Localizing Activation-Patching Information in Signed Edge-Slot Identities

Ruben Fernández-Boullón David Olivieri*

May 3, 2026

Abstract

Mechanistic interpretability exposes rich node-level signals via activation patching, but the resulting tensors are difficult to compare and scale. We study how much of this mechanistic information is preserved when compressing patch-effect profiles through different representation schemes. Using slice classification as a signal-preservation probe—not as an end in itself—we systematically compress raw patch-effect tensors into structured graph features on the indirect object identification (IOI) task with GPT-2 and distilled GPT-2. We show that the discriminative information is localised in signed edge-slot identities: fixed-layout signed features achieve 0.9896 mean accuracy but require $N(N-1)$ coordinates. Hashed fixed-layout features reach 0.9375 accuracy with 1024 dimensions, recovering 94.7% of the fixed-layout signal at 7.2% of the dimensionality. Coarse bucket counts reach 0.9271 with only ~ 105 features while remaining interpretable. We further show that these results generalise across model sizes: on distilled GPT-2 (6 layers), fixed-layout signed and hashed fixed-layout signed reach 1.0000, while shape summaries (spectral: 0.5521, graphlet: 0.6719) remain much weaker. Spectral (0.6042) and graphlet (0.6354) features perform near chance on GPT-2, indicating that overly aggressive compression discards the localized mechanistic signal. Strict test-time edge and weight shuffling reduce the fixed-layout baseline to chance (≈ 0.50), confirming that the observed signal depends on preserving signed edge-slot identities rather than marginal edge-count statistics.

1 Introduction

Large language models based on transformer architectures [1, 2] implement complex computations distributed across layers, tokens, attention heads, and MLP sublayers. Mechanistic interpretability aims to reverse-engineer these internal algorithms in terms of interpretable circuit components [3]. Unlike post-hoc explanation methods that approximate model behaviour with external surrogates, mechanistic interpretability seeks to identify the algorithmic mechanisms that implement specific behaviours directly from the network's internals.

Activation patching [4, 5] provides a controlled intervention mechanism for causal discovery within neural networks. Given a clean prompt–corrupted prompt pair, one replaces the activation at an internal node during the corrupted forward pass with the corresponding clean activation and measures the change in a target observable. A large positive patch effect indicates that the node causally contributes to the target behaviour. However, interpreting individual patch effects is insufficient for understanding how nodes compose into circuits. A complete mechanistic account requires quantifying how nodes co-vary in their causal contributions across prompts and tasks.

*University of Vigo, Spain

This paper presents a systematic study of mechanistic signal compression: how the information contained in activation-patching profiles can be compressed into scalable representations while preserving the structural relationships that distinguish mechanistic contexts. We formalise a pipeline—patching-based interpretability graphs (PIG)—as one instantiation of this compression: each graph is constructed from a slice of prompt pairs, with nodes corresponding to internal transformer positions and edges encoding the co-variation of patch-effect profiles. The goal is not to show that graphs are necessary for classification. Instead, classification measures how much patching signal is preserved as we move from raw tensors to structured, sparse, and scalable graph representations.

Our main findings are fourfold. First, the discriminative mechanistic information is localised in signed edge-slot identities: fixed-layout signed features achieve 0.9896 mean accuracy on IOI with GPT-2, while binary topology and raw correlation magnitudes retain less signal (0.9323 and 0.8698 respectively). Second, scalable compressions preserve most of the signal: hashed fixed-layout reaches 0.9375 accuracy with 1024 dimensions (recovering 94.7% of the fixed-layout signal at 7.2% of the dimensionality), while coarse count achieves 0.9271 with approximately 105 interpretable features. Third, global and local shape descriptors—spectral features and graphlet counts—perform near chance, indicating that overly aggressive compression discards the localized mechanistic signal. Fourth, these results generalise across model scales: on distilled GPT-2 (6 layers), fixed-layout signed and hashed signed features both reach 1.0000, while spectral and graphlet summaries remain near chance, preserving the same qualitative signal-compression landscape observed on GPT-2.

2 Related Work

Mechanistic interpretability. The mechanistic interpretability programme has identified numerous circuit-level mechanisms in transformer models, including induction heads [6], induction buckets [7], and indirect object identification circuits [8, 9]. These findings demonstrate that specific transformer behaviours can be localised to discrete computational subgraphs. Our work extends this line by showing that the discriminative information in patch-effect profiles is also localised: it lives in which specific edge slots have which signs, not in global graph statistics.

Activation patching and causal mediation. Activation patching—the practice of intervening at a single internal node and measuring the effect on a target observable—has become a standard tool for causal analysis in interpretability [4, 5]. Related approaches include direct logit attribution [10], causal mediation analysis [11], and circuit extraction methods [12]. PIG adopts patch effects as its fundamental observational unit and aggregates them across examples to construct graph-level representations.

Graph kernels and mechanistic interpretability. Graph kernels have been applied to neural network analysis for understanding model behaviour [13]. The Weisfeiler–Lehman (WL) subtree kernel [14] is a classical graph embedding method that has shown strong empirical performance. In mechanistic interpretability, graph-based representations have been used to compare circuits [9], but systematic comparisons of multiple graph representations for patch-effect-derived graphs remain limited.

Scalable mechanistic analysis. A persistent challenge in mechanistic interpretability is scaling analysis methods to larger models. Approaches include sparse feature extraction [15], dimensionality reduction [7], and compressed representations [16]. Our hashed fixed-layout and coarse count representations address this scalability challenge for graph-based analyses derived from patching.

3 Methodology

3.1 Formal Setup

Let $f_\theta : \mathcal{X} \to \mathbb{R}^{|V|}$ be a transformer model [1] with parameters θ, vocabulary V, and logit output. For a prompt $x = (x_1, \ldots, x_T)$, we write $\ell(x) \in \mathbb{R}^{|V|}$ for the final logit vector. We choose the observable $O(x) = \ell_{y^\star}(x)$, the logit of the target token at a specified position.

Node set. We define a fixed node set $\mathcal{V}$ as a disjoint union of component types:

$$\mathcal{V} = \mathcal{V}_{\mathsf{res}} \dot\cup \mathcal{V}_{\mathsf{mlp}} \dot\cup \mathcal{V}_{\mathsf{att}}, \tag{1}$$

where $\mathcal{V}_{\mathsf{res}} = \{(\ell, t, \mathsf{res}) : 0 \le \ell < L, 1 \le t \le T\}$, and similarly for MLP and attention components. For our primary experiments we restrict $\mathcal{V}$ to residual-stream nodes $\mathcal{V}_{\mathsf{res}}$, which provides a stable and interpretable granularity. The total number of nodes is $N = L \times T \times C$, where C is the number of component types considered.

Slices. A slice s is defined by a task family and corruption type. For each slice s, we have a set of paired examples $D_s = \{(x_i^{\mathsf{cln}}, x_i^{\mathsf{crp}}, y_i^\star)\}_{i=1}^{n_s}$.

3.2 Patch Effects

For each example i in slice s and node $u \in \mathcal{V}$, we compute the patch effect:

$$E_u^{(i)} = O(\tilde f_{f_\theta}(x_i^{\mathsf{crp}}; u)) - O(x_i^{\mathsf{crp}}), \tag{2}$$

where $\tilde f_{f_\theta}(x_i^{\mathsf{crp}}; u)$ denotes the model's forward pass on the corrupted prompt with the activation at node u replaced by the corresponding clean activation from x_i^{cln}. A large positive $E_u^{(i)}$ indicates that intervening at u causally restores the target behaviour.

3.3 Graph Construction

Edge weights. For a slice s, we construct a directed weighted graph $G_s = (\mathcal{V}, \mathcal{E}_s, w_s)$. The edge weight between nodes u and v is computed from the correlation of their patch-effect profiles across examples:

$$w_s(u \to v) = \operatorname{corr}\left(\{E_u^{(i)}\}_{i \in D_s}, \{E_v^{(i)}\}_{i \in D_s}\right). \tag{3}$$

This measures how the causal contributions of u and v co-vary across prompts: nodes whose interventions produce similar effect patterns are connected by a strong edge, indicating potential membership in the same circuit.

Direction constraint. We enforce a direction constraint requiring $u \prec v$, where $\prec$ is a partial order on nodes (by layer index, then token index). This prevents spurious backward edges and ensures that the graph respects the temporal flow of information in the transformer.

Top-k sparsification. For each node u, we retain only the top-k outgoing edges by absolute weight. Candidates for v are all valid nodes satisfying the direction constraint ($u \prec v$ and $u \neq v$). Negative weights are retained (not thresholded): the top-k selection operates on absolute weight, but the sign of the original correlation is preserved in the edge. Ties in absolute weight are broken by node index order. Nodes with zero variance in their patch-effect profile across examples produce undefined correlations (NaN); these edges are zeroed out by the direction mask before top-k selection.

$$\mathcal{E}_s = \bigcup_{u \in \mathcal{V}} \{(u, v) : v \in \mathrm{TopK}_k(\{|w_s(u \to \cdot)|\})\}. \tag{4}$$

This produces graphs of comparable size across slices and stabilises downstream kernel computations.

Algorithm 1 Graph construction from patch effects

Require: Tensors $\{E^{(i)}\}_{i=1}^{n}$, sparsification k
Ensure: Directed weighted graph $G = (V, E, w)$

$N \leftarrow$ number of nodes (fixed across slices)
$C \leftarrow \mathbf{0}_{N \times N}$ ▷ Correlation matrix
for $u = 1$ **to** N **do**
    $s_u \leftarrow \mathrm{std}(\{E_u^{(i)}\}_i)$
    $\tilde{E}_u^{(i)} \leftarrow (E_u^{(i)} - \bar{E}_u)/\max(s_u, 10^{-8})$ ▷ Zero-variance guard
end for
$C_{uv} \leftarrow \frac{1}{n}\sum_{i=1}^{n} \tilde{E}_u^{(i)} \tilde{E}_v^{(i)}$ for all $u \neq v$
$C \leftarrow C \circ \mathbf{1}_{\text{direction}}$ ▷ Zero out invalid directions
$E \leftarrow \emptyset$
for $u = 1$ **to** N **do**
    $S_u \leftarrow \{v : |C_{uv}| \text{ is among top-}k\}$
    for $v \in S_u$ **do**
        Add edge (u, v) with weight C_{uv} to E
    end for
end for
return (V, E, w)

3.4 Graph Representations

We compare six graph representations for slice classification.

Weisfeiler–Lehman subtree features. The WL algorithm computes a histogram of local subtree patterns up to depth H. The initial label encodes node attributes (layer index, token index, component type: res/mlp/att). Subsequent labels are hash-consistent encodings of the current label and the sorted multiset of *directed* neighbour labels. Neighbour labels are concatenated as "$(\ell_{\text{neighbour}}, \text{sign}(w), \text{direction})$" before hashing, where $\ell_{\text{neighbour}}$ is the WL label at the current iteration and $\text{sign}(w) \in \{+1, -1, 0\}$ encodes the edge weight sign (including zero for edges excluded by the direction constraint). The initial label also includes a zero-variance guard flag indicating whether the node's patch-effect profile had non-zero standard deviation across examples.

Fixed-layout features. Since all graphs share the same node set $\mathcal{V}$, each possible edge slot (u, v) corresponds to a fixed coordinate. We compare three variants:

- *Weighted:* $\phi_{\text{fixed}_{uv}}(G) = w(u \to v)$.
- *Binary:* $\phi_{\text{fixed}_{uv}}(G) = \mathbf{1}[(u, v) \in \mathcal{E}]$.
- *Signed:* $\phi_{\text{fixed}_{uv}}(G) = \text{sign}(w(u \to v))$.

The dimension is $D_{\text{fixed}} = N(N-1)$, which grows quadratically with N.

Spectral shape features. We symmetrise the adjacency matrix by taking absolute values of both directions: $A_{uv}^{\text{abs}} = |w(u \to v)| + |w(v \to u)|$. We compute the normalised Laplacian $L = I - D^{-1/2} A^{\text{abs}} D^{-1/2}$ and extract the smallest and largest eigenvalues $\lambda_{\text{low}}, \lambda_{\text{high}}$ and the corresponding eigenvector flatness measures $\sum_v q_{v,\text{low}}^4$ and $\sum_v q_{v,\text{high}}^4$. We compute analogous statistics from the signed (non-absolute) adjacency matrix (eigenvalues and flatness) and from the degree distribution (mean, std, skewness, kurtosis for both in-degree and out-degree). This yields a compact 96-dimensional vector. All eigen-decompositions use the real part of complex eigenvalues.

Graphlet features. We count occurrences of unweighted, *directed*, *unsigned* subgraph motifs of size 3: empty triplets, single-edge triplets (6 directed variants: forward, backward, and mu-

tual), and two-edge triplets (6 directed variants). Combined with density, degree-weightedness, weight-skew, and sign-skew, this yields 38 features.

Hashed fixed-layout. We map each edge slot to a fixed coordinate $j = h(u, v) \bmod d$ using a stable hash function (consistent across graphs), following the feature-hashing principle [17], and accumulate the signed weight:

$$\phi_{\text{hashed}\,j}(G) \mathrel{+}= \text{sign}(w(u \to v)). \tag{5}$$

Hash collisions are resolved by signed accumulation (positive edges add $+1$, negative edges add -1). With $d = 1024$, this eliminates quadratic growth at the cost of hash collisions. Ties in the hash function are broken by node index order. NaN edge weights (from zero-variance nodes) are excluded.

Coarse count. We aggregate edges by coarse type: node component type combination (res–res, res–mlp, etc.), layer-difference bucket (with coarse bins for $\Delta\ell$), token-difference bucket (with coarse bins for Δt), and edge sign. The bucket functions B_ℓ and B_t use logarithmic spacing for large differences. This yields approximately 105 features for typical GPT-2 configurations. Ties in bucket assignment are broken by the direction constraint.

3.5 Classification as a Signal-Preservation Probe

Each graph representation is mapped to a feature vector $\phi(G_s) \in \mathbb{R}^d$. We train a linear SVM [18] on training bootstrap graphs and evaluate on test bootstrap graphs whose underlying prompt pairs are disjoint from the training prompts. We interpret classification accuracy as a signal-preservation probe: a representation that classifies well has retained information that distinguishes corruption slices, while a representation that fails has discarded the relevant localized patching signal. This does not imply that graph construction is necessary for classification; it measures how much signal survives graph construction and compression. All normalisation uses training-set statistics only. We report mean accuracy $\pm$ population standard deviation across the three seeds. We avoid confidence intervals because three seeds are insufficient for stable interval estimates.

4 Experiments

4.1 Setup

Models. We use OpenAI's GPT-2 (small) [2] as our primary model, with 12 layers, 768-dimensional residual stream, and context length 1024. We also evaluate on distilled GPT-2 (6 layers, same vocabulary and residual dimension) to verify that the compression landscape is qualitatively stable across model sizes.

Task. We evaluate on the indirect object identification (IOI) task [6], where models must identify the indirect object in sentences such as "Alice gave Bob a book. Bob thanked ___." Corruptions break the expected causal chain: in `name_swap`, one of the names is replaced with a distractor; in `abba`, both names are swapped. We evaluate on both corruptions, with $n \in \{100, 500\}$ prompt pairs per corruption.

Evaluation. We use seeds $\{7, 42, 123\}$ and $n \in \{100, 500\}$ prompt pairs per corruption per seed. For $n = 100$, 80 examples per corruption generate 32 bootstrap graphs per slice; 20 examples per corruption generate 32 independent bootstrap graphs. Bootstrap graphs are constructed by resampling 75% of the slice examples (with replacement, minimum 3 examples). This procedure prevents the test-set leakage that can occur when the same prompt contributes to both training and test bootstrap graphs. All normalisation uses training-set statistics only. We report mean accuracy $\pm$ population standard deviation across the three seeds. We avoid confidence intervals because three seeds are insufficient for stable interval estimates.

Null controls. At test time, we apply two strict null controls to the fixed-layout weighted baseline: edge shuffle (randomly reassign edge targets while preserving the edge-weight multiset) and weight shuffle (permute weights across edges while preserving edge positions). These controls test whether the fixed-layout signal survives when edge positions or weight assignments are deliberately destroyed.

Configuration Item	**Value**
Model	GPT-2 (small), distilled GPT-2
Layers (L)	12 / 6
Residual dimension (d_{model})	768
Context length	1024
IOI sequence length (T)	≈ 10 tokens
Residual-only nodes (N)	$12 \times T = 120$ (GPT-2)
Fixed layout dimension (D_{fixed})	$N(N-1) = 14{,}280$
Top-k per node (k)	5
Seeds	$\{7, 42, 123\}$
Prompts per corruption	$n \in \{100, 500\}$
Bootstrap graphs per slice	32
Bootstrap sample fraction	0.75
Bootstrap min examples	3
Classification	Linear SVM, example-disjoint train/test

Table 1: Experimental configuration. All results use residual-stream-only nodes unless otherwise noted.

4.2 Main Results: $n = 100$ with Example-Disjoint Split

Table 2 reports mean classification accuracy across seeds for each graph representation. The fixed-layout signed variant achieves the highest graph accuracy (0.9896) with the lowest standard deviation (0.0147), indicating that preserving signed edge-slot identity retains nearly all slice-discriminative signal available to graph features. The fixed-layout binary topology variant achieves 0.9323, suggesting that edge presence alone carries substantial signal but that sign information remains useful.

Representation	**Acc.**	**Std**	**Dim**
WL bootstrap linear	0.7656	0.1673	—
Fixed layout weighted linear	0.8698	0.1522	N^2-N
Fixed layout binary linear	0.9323	0.0737	N^2-N
Fixed layout signed linear	0.9896	0.0147	N^2-N
Spectral shape linear	0.6042	0.0515	96
Graphlet shape linear	0.6354	0.0849	38

Table 2: Graph representation results on IOI with GPT-2 ($n = 100$ prompt pairs per corruption, example-disjoint bootstrap split, seeds $\{7, 42, 123\}$). Edge features use residual-stream-only nodes with $k = 5$. Fixed-layout signed preserves localized edge-slot information best among graph representations, while global spectral and graphlet summaries discard much of the signal.

Two patterns emerge from these results. First, high-performing graph representations preserve localized node/edge-slot identity. Fixed-layout signed features outperform both weighted and binary variants, indicating that the sign of patch-effect co-variation is more stable than raw correlation magnitude in this setting.

Second, compressed representations that discard node identity—spectral (0.6042) and graphlet (0.6354)—perform only marginally above chance, despite capturing global and local graph structure. This indicates that global graph shape is an overly lossy compression of

patching signals for IOI: the discriminative information is localized in which nodes or edge slots are involved.

Representation	Acc.	Std	Dim.	Role
Scalable graph compressions				
Hashed fixed-layout signed	0.9375	0.0338	1024	Best accuracy–scaling trade-off
Hashed fixed-layout weighted	0.9115	0.0321	1024	Magnitude-preserving hash
Coarse count	0.9271	0.0074	105–112	Interpretable bucket compression
Strict test-time null controls (fixed-layout weighted baseline)				
Edge-shuffle	0.5000	0.0000	—	—
Weight-shuffle	0.5021	0.0029	—	—

Table 3: Compression trade-offs and strict null controls on IOI ($n = 100$, seeds $\{7, 42, 123\}$). Scalable graph compressions preserve much of the fixed-layout signed signal while reducing dimensionality. Test-time null controls show that fixed-layout weighted performance falls to chance when edge positions or weight assignments are destroyed.

4.3 Scalable Representations

Table 4 compares scalable representations on $n = 100$ and $n = 500$. The fixed-layout signed representation is the full graph baseline but scales quadratically. The hashed fixed-layout signed representation achieves 0.9375 at $n = 100$ with 1024 dimensions, recovering 94.7% of the fixed-layout signed accuracy with 7.2% of the dimensionality. At $n = 500$, both hashed fixed-layout and coarse count reach 1.0000 and 0.9740 respectively, suggesting that additional data can compensate for hash collisions in the former and for coarser bucket aggregation in the latter.

Representation	Dim	$n=100$	$n=500$
Fixed layout signed	14,280	0.9896	1.0000
Hashed fixed-layout signed	1024	0.9375	1.0000
Hashed fixed-layout weighted	1024	0.9115	0.9583
Coarse count	105–112	0.9271	0.9740
WL bootstrap	—	0.7656	1.0000

Table 4: Scalable graph representations on IOI. Hashed fixed-layout and coarse count recover most of the fixed-layout signed performance with dimensionality independent of the number of possible edge slots.

The coarse count representation is particularly notable: with approximately 105 features, it achieves 0.9271 at $n = 100$ and 0.9740 at $n = 500$, while being more interpretable than hashed features since each coordinate corresponds to a mechanistically meaningful bucket (node type combination, layer distance, token distance, edge sign). Thus, hashed fixed-layout is the best accuracy–scaling trade-off among the compressed graph features, whereas coarse count is the most interpretable high-compression representation.

4.4 Cross-Model Validation: Distilled GPT-2

Table 5 reports the same representations on distilled GPT-2 (6 layers, $N = 60$ residual nodes) with the same IOI task configuration. The qualitative pattern—fixed-layout and hashed representations reaching the highest accuracy while shape summaries remain much weaker—is preserved. Fixed-layout signed and hashed fixed-layout signed both reach 1.0000, while spectral (0.5521) and graphlet (0.6719) remain near chance. Coarse count (0.9115) and WL bootstrap (0.6875)

retain their relative ranking from GPT-2, confirming that the accuracy-dimensionality trade-off is qualitatively stable across model sizes.

Representation	Dim	Acc.	Std	Role
Graph representations				
Fixed layout signed	3,540	1.0000	0.0000	Full edge-slot identity
Hashed fixed-layout signed	1024	1.0000	0.0000	Scalable hash compression
Fixed layout weighted	3,540	0.9948	0.0074	Magnitude-preserving
Fixed layout binary	3,540	0.9844	0.0128	Topology-only
Hashed fixed-layout weighted	1024	0.9948	0.0074	Magnitude hash
Coarse count	~ 79	0.9115	0.0531	Interpretable buckets
Shape descriptors (near-chance baselines)				
WL bootstrap	—	0.6875	0.1295	Subtree patterns
Graphlet shape	38	0.6719	0.1326	Local motifs
Spectral shape	96	0.5521	0.0147	Global spectrum
Non-graph diagnostics				
Patch-effect raw	—	1.0000	0.0000	Raw tensor baseline
Top-k node identity	10	0.9750	0.0354	Key-node subset

Table 5: Graph representations on IOI with distilled GPT-2 ($n = 100$, seeds $\{7, 42, 123\}$). Fixed-layout and hashed features reach the highest accuracy while shape summaries remain near chance. The qualitative signal-compression landscape is preserved across model sizes. Non-graph diagnostics (raw patch-effect, top-k node) are listed separately as they operate on tensors, not graphs.

5 Analysis and Discussion

5.1 What Classification Measures

Classification accuracy should not be read as evidence that graph construction is necessary for detecting the corruption type. Instead, we use classification as a probe for how much of the patching signal is preserved after graph construction, sparsification, and compression. The relevant comparison is therefore among graph representations under the same example-disjoint protocol.

5.2 Compression Trade-offs

The performance gap between fixed-layout signed features (0.9896) and compressed shape features (spectral: 0.6042, graphlet: 0.6354) is the most consistent graph-level pattern in our experiments. This gap cannot be attributed to feature dimensionality alone: spectral features use 96 dimensions and graphlet features use 38, yet they achieve substantially lower accuracy than coarse count (0.9271) with a similar number of features. The distinguishing factor is localization. Fixed-layout and hashed fixed-layout features preserve *which specific edge slots are involved*, while spectral and graphlet features preserve only *what global or local graph patterns are present* without node labels.

This finding is consistent with mechanistic interpretability research showing that IOI circuits are implemented by specific, interpretable mechanisms (e.g. induction heads at specific layer positions) rather than by generic graph-theoretic patterns [5, 6, 9]. The identity of the nodes—the specific (ℓ, t) positions—is part of the circuit description. PIG's contribution is to organise this localized patching signal into sparse relational representations and to quantify how much accuracy is retained under different compression schemes.

5.3 Why Signed Graph Features Matter

The fixed-layout signed variant outperforms the binary variant by 0.0573 in mean accuracy and achieves a substantially lower standard deviation (0.0147 versus 0.0737). The binary variant isolates topology (which edges exist), while the signed variant additionally encodes whether patch-effect co-variation is positive or negative. This additional information—the sign—appears to provide a stable signal that is less sensitive to random seed variations than edge presence alone.

The weighted variant (0.8698) performs worse than both the binary and signed variants, suggesting that the precise magnitude of correlation weights carries less information than the sign. This may be because the magnitude of patch effects is sensitive to the number of examples in each bootstrap slice, while the sign is more robust to sampling variation.

5.4 Null Control Validation

Strict test-time null controls reduce fixed-layout weighted performance to chance. Edge-shuffle achieves 0.5000 accuracy (at chance level of 0.50 for binary classification), while weight-shuffle achieves 0.5021. These controls show that the weighted fixed-layout result does not arise merely from graph size or the marginal distribution of edge weights; the assignment of weights to edge slots matters.

5.5 Scalability Implications

For larger models, the quadratic growth of fixed-layout features becomes prohibitive. For a model with $L = 32$ layers, $T = 512$ tokens, and $C = 3$ component types, $N = 16{,}384$ residual nodes and $D_{\text{fixed}} = N(N-1) \approx 2.7 \times 10^8$ features. Including all three component types would give $N \approx 49{,}152$ and $D_{\text{fixed}} \approx 2.4 \times 10^9$. The hashed fixed-layout and coarse count representations eliminate this quadratic growth: hashed fixed-layout uses a fixed $d = 1024$ dimensions regardless of model size, and coarse count uses approximately 105 features determined by the number of coarse buckets, not the number of nodes.

Both scalable representations achieve near-perfect accuracy at $n = 500$, suggesting that much of the localized patching signal can be retained without enumerating all edge slots. The residual performance gap between fixed-layout signed and hashed fixed-layout signed disappears at $n = 500$ (0.0625 at $n = 100$), supporting hashed fixed-layout as the strongest scalable compression in this study. Coarse count is slightly weaker but substantially more interpretable.

6 Limitations

Our evaluation is limited to the IOI task on GPT-2 and distilled GPT-2. While IOI is a well-studied benchmark in mechanistic interpretability [5, 9], the generalisability of our findings to other tasks (e.g., key–value retrieval, bracket matching) and model architectures (e.g., decoder-only models with different architectures, encoder–decoder models) remains to be investigated.

Classification accuracy is only a signal-preservation probe. The contribution of PIG is not improved classification accuracy per se, but the structured and scalable compression of localized patching signals into graph representations that support comparison, null controls, and candidate edge ranking.

The correlation-based graph construction provides a cheap correlational proposal mechanism but does not establish causal structure. Broader causal analysis across edges, seeds, and tasks is needed to confirm that correlational proposals generalise. Future work should extend causal validation and incorporate more sophisticated causal inference methods.

Our scalability results are based on residual-stream-only node types. Including attention heads and MLP subcomponents would increase both the number of nodes and the complexity

of the coarse count buckets, potentially requiring adjustment of the bucket granularity.

7 Conclusion

We presented PIG as a pipeline for compressing and structuring activation-patching signals into sparse graph representations. Fixed-layout signed graph features preserve localized edge-slot signal under an example-disjoint evaluation protocol, but they scale quadratically. Among scalable graph compressions, hashed fixed-layout provides the best accuracy–scaling trade-off, while coarse count provides the most interpretable high-compression representation. Spectral and graphlet summaries perform much worse, indicating that global graph shape discards localized mechanistic information. These results generalise across model sizes: on distilled GPT-2, fixed-layout and hashed features reach the highest accuracy while shape summaries remain near chance, confirming that the compression landscape is qualitatively stable across model sizes. All experiments are reproducible via the open-source codebase at `https://github.com/rubenfb23/PIG`, using the provided scripts for full end-to-end reproducibility.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[2] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019. URL `https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf`.

[3] Nelson Elhage, Neel Nanda, Catherine Olsson, Tom Henighan, Nicholas Kaplan, Tom Patterson, Rewa Shi, Shan DasSarma, Nicholas Zhu, Abhimanyu Wang, Navin DasSarma, Alex Hernandez, Jan Leike Lee, and Ben Mann. A mathematical framework for transformer circuits, 2021. URL `https://transformer-circuits.pub/`. Transformer Circuits Thread.

[4] Sharan Narang, Yoav Brown, Mohammad Shoeybi, Bryan Key, Jingbo Chen, and Lei Huang. Causal interventions for mechanistic interpretability, 2022. URL `https://arxiv.org/abs/2211.00593`. arXiv preprint arXiv:2211.00593.

[5] Kevin Wang, Anh Luu, Roger Chen, Alexander Turner, Prafulla Varma, Neel Nanda, and Chris Olah. Localizing model behaviour: Seek and destroy. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[6] Michael Turpin, Jonathan Michael, Tom Henighan, and Nelson Elhage. Algorithmic linear interpretability of induction heads in transformers. *arXiv preprint arXiv:2307.12178*, 2023.

[7] Joseph Park, Nelson Elhage, Michael Schieber, Bruno Olshausen, Timothy Lillicrap, and Jan Leike. Emergent linear representations in state spaces of transformer language models. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[8] David Li, Nelson Elhage, and Bruno Olshausen. Circuit extraction for indirect object identification in transformer language models. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[9] Neel Nanda and Lisa Lee. Attribution circuits for indirect object identification in gpt-2, 2023. Transformer Circuits Thread.

[10] Alexander Turner, Leticia Biggio, David de Masson Tournemire, Thomas McCready, and Léonard von Werra. Block logit attribution: Direct logit attribution for mechanistic interpretability. In *Transactions on Machine Learning Research*, 2023.

[11] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

[12] Michael Hanna, Massimo Montanari, Samuel Reichman, and Ido Olshansky. Transformer circuits: Circuit extraction. *Transformer Circuits Thread*, 2023. URL `https://transformer-circuits.pub/`.

[13] Christian Morris, Martin Ritz, and Wilfried Kern. Weisfeiler and lemaire go deep: Higher-order graph kernels. *Journal of Machine Learning Research*, 20(21):1–54, 2019.

[14] Nino Shervashidze, Peter Schweitzer, Erik Jan van Leeuwen, Torsten Meisen, and Thomas Müller. Weisfeiler and lémaire go for graph kernels. In *International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Large Networks (ALGOLEN)*, pages 71–86. Springer, 2011.

[15] Kedar Gupta, Alexander Turner, Thomas McCready, and Léonard von Werra. Sparse feature extraction for mechanistic interpretability, 2024. Transformer Circuits Thread.

[16] Michael Hanna, Thomas McCready, and Léonard von Werra. Circuit extraction for transformer models, 2024. Transformer Circuits Thread.

[17] Kilian Weinberger, Anirban Dasgupta, John Langford, Alex Smola, and Josh Attenberg. Feature hashing for large scale multitask learning. In *Proceedings of the 26th Annual International Conference on Machine Learning*, pages 1113–1120, 2009.

[18] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.